



Department of Computer Science

Lab Manual

of

MACHINE LEARNING

MCA521-4

Class Name: IV MCA

Master of Computer Application

2026

Prepared by: Sharon Mathew

Verified by:

Dr. Rupali Sunil Wagh Dr. Helen K Joy Dr. Shivangi Singh

Department Overview

Department of Computer Science of CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape nation's destiny. The training imparted aims to prepare young minds for the challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field.

Vision

The Department of Computer Science endeavours to imbibe the vision of the University “**Excellence and Service**”. The department is committed to this philosophy which pervades every aspect and functioning of the department.

Mission

“To develop IT professionals with ethical and human values”. To accomplish our mission, the department encourages students to apply their acquired knowledge and skills towards professional achievements in their career. The department also moulds the students to be socially responsible and ethically sound.

Introduction to the Programme

Master of Computer Applications (MCA) is a post-graduate programme of CHRIST (Deemed to be University) and approved by the All India Council of Technical Education (AICTE). It is a two-year programme spread over six trimesters to bridge the chasm between computer studies and applications, bringing within reach of enthusiastic youngsters an excellent group of experienced and dedicated staff and experts, and an enviable infrastructure. MCA at CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape the nation's destiny. The training programme aims to prepare young minds for challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field. The Department is committed to the motto “Excellence and Service,” and this philosophy pervades every aspect of its functioning.

Programme Objective

This programme is conceptualised and designed based on the strong commitment of the department to provide better quality education to the students. The principal objectives of this course are to:

- Heighten technological awareness
- Train future industry specialists
- Encourage effective software development
- Provide research training
- Involve students in innovative research
- Produce graduates with a two-year professional education in Computer Science with technical, professional, and communications skills.

Ethics and Human Values

1. Only proprietary or open-source software would be used for academic teaching and learning purposes.
2. Copying of programs from the internet, friends or from other sources is strictly discouraged since it impairs development of programming skills.
3. Unique Practical (Domain based) exercises ensure that the students don't get involved in code plagiarism.
4. Projects undertaken by students during the course are done in teams to improve collaborative work and synergy between team members.
5. Projects involve modularization which initiates students to take individual responsibility for common goals.
6. Passion for excellence is promoted among the students, be it in software development or project documentation.
7. Giving due credit to sources during the seminar and research assignment is promoted among the students
8. The course and its design enforce the practice of good referencing technique to improve the sense of integrity.
9. Courses involving group discussions and debates on ethical practices and human values are designed to sensitize the students in dealing with customers and members within the Organization.

Programme Outcomes:

- P01: Foundation Knowledge: Apply knowledge of mathematics, programming logic, and coding fundamentals for solution architecture and problem-solving.
- P02: Problem Analysis: Identify, review, formulate, and analyse problems primarily focussing on customer requirements using critical thinking frameworks.
- P03: Development of Solutions: Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.
- P04: Modern Tool Usage: Select, adapt, and apply modern computational tools such as the development of algorithms with an understanding of the limitations including human biases.
- P05: Individual and Teamwork: Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.
- P06: Project Management and Finance: Use the principles of project management such as scheduling, and work breakdown structure and be conversant with the principles of Finance for profitable project management.
- P07: Ethics: Commit to professional ethics in managing software projects with financial aspects.
- Learn to use new technologies for cyber security and insulate customers from malware
- P08: Life-long learning: Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

Programme Educational Objectives(PEO's)

- PEO1: Develop innovative and effective software solutions through research that addresses real-world challenges, grounded in a commitment to continuous learning and adaptability.
- PEO2: Advance the knowledge and practices in the field of computer applications through lifelong learning and rigorous research, supporting professional growth and academic excellence.
- PEO3: Exhibit ethical leadership, social responsibility to contribute and enhance community well-being by innovating solutions.

MCA521-4 – MACHINE LEARNING

Course Name: MACHINE LEARNING Code: MCA521-4

Total Hours: 75

L + P + T: 3 + 4 + 0

Max Marks: 100

Credits: 4

Course Type: Core Course

Course Category: Theo&Prac

Course Description

Machine Learning is a key area in computer applications that focuses on building models capable of making predictions or informed decisions based on data. A strong understanding of machine learning enables students to analyze and interpret complex datasets, develop predictive models, and extract meaningful insights. This course covers a wide range of machine learning concepts and techniques, including supervised and unsupervised learning. It provides a solid theoretical foundation while emphasizing practical, hands-on experience with algorithms and real-world data. The course equips students with the knowledge and skills required to apply machine learning techniques across various domains using modern tools and methodologies.

Course Objectives

This course enables students to understand the fundamental concepts of Machine Learning, including its core principles, terminology, and methodologies. Students will learn to differentiate between various types of learning algorithms such as supervised, unsupervised, and reinforcement learning, and recognize their appropriate applications.

Course Outcomes

After successful completion of the course, students will be able to:

- Explain the fundamental principles and scope of Machine Learning
- Implement modelling practices in machine learning for better generalisation
- Interpret predictive modelling results and performance of supervised machine learning techniques
- Assess the outcomes and effects of unsupervised machine learning processes
- Deploy robust machine learning models for interdisciplinary applications

Unit-1: INTRODUCTION TO MACHINE LEARNING Teaching Hours: 15 Hours

Introduction to AI – Knowledge Representation – Data – Representation of Data, Data Processing, Data Storage, Data Processing, Types of Data – Exploratory Data Analysis, Visualisations and Interpretation, Outliers

Machine Learning: Definition, Objectives, Components, Features, Applications – Traditional Programming v/s Machine Learning, Types of Machine Learning – Predictive Models, Statistical Inferences – Applications of Machine Learning

Challenges in ML: Insufficient Quantity of Data, Non-representative and Poor-Quality Data, Irrelevant Features, Estimation Estimation, Reducing Human Biases in Model Building, Ethical Use of Technology

Lab Exercises:

1. Data Preprocessing and visualisation
2. Data exploration and inferences

Unit-2: PROCESSES IN MACHINE LEARNING Teaching Hours: 15 Hours

Data Preparation and Model Selection: Effect of Data Preparation: Encoding, Nominal and Ordinal Representation, Feature Transformation and Feature Engineering, Generalisation, Normalisation, Sampling, Using Saved Weights, Model Selection Strategies: Overfitting, Underfitting, Bias-Variance Trade-off, Testing and Validation: Performance measures - Confusion matrix, Precision-Recall Trade off, F1 Score, ROC Curve, AUC, Error Analysis, Model Parameters, Hyperparameters, Hyperparameter Tuning, Cross Validation, Decision Functions: Introduction to Supervised Machine Learning Methods, Decision Functions, Decision Thresholds, Distance Measures, Hyperplanes, Regression & Classification Problems, Loss and Cost Functions, Linear Regression using OLS, Multi-class vs Multi-label, KNN Classification

Lab Exercises:

3. Saving weights of a generalised model.
4. Regression and Classification Evaluation Metrics: Illustration through Linear Regression and KNN Classification

Unit-3: SUPERVISED LEARNING ALGORITHMS**Teaching Hours: 15****Hours**

Learning through Optimisation: Gradient Descent, Linear Regression, Logistic Regression
Computational Learning: Decision Trees, Naive-bayes classification, Support Vector Machines, Ensemble Learners: Learning, Voting, Bagging, Stacking, Boosting, Random Forests, GridSearchCV

Lab Exercises:

5. Regression through Gradient Descent
6. Comparison of Different Classifiers
7. Illustration of Ensemble Learning Methods

Unit-4: UNSUPERVISED LEARNING AND DIMENSIONALITY REDUCTION Teaching Hours: 15 Hours

Introduction: Types of Unsupervised Learning, Challenges in Unsupervised Learning, Concept of Cost Functions in Unsupervised Learning, Clustering Methods: Distance based, Centroid Based, Elbow method to find Optimal Number of Clusters, Silhouette Method, K-Means Clustering, Hierarchical Clustering: Agglomerative and Bottom Up, Applying Supervised Learning after Clustering, Dimensionality Reduction Methods: Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA)

Lab Exercises:

8. KMeans and Elbow Methods
9. Hierarchical Clustering and Dendrograms
10. Dimensionality Reduction
11. Image Compression using Dimensionality Reduction.

Unit-5 Teaching Hours: 15 Hours**MACHINE LEARNING IN REAL-WORLD**

Emerging Techniques: Role of ML in attaining Sustainable Development Goals, Time Series Preprocessing, Blackbox Methods: Neural Networks & Deep Learning, Reinforcement Learning/QLearning, Self Supervised Learning, Quantum Machine Learning, Keras and Tensorflow for designing Multi Layer Perceptron. Case Studies: CNN, RNN, Advances in Deep Learning, Natural Language Processing, Real Time Systems, Computer Vision, Robotics, Expert Systems, Generative Networks, Large Language Models. Model Deployment: Model Deployment: Connecting Models to User Interface, Deployment of ML Models.

Lab Exercises:

12. Training a Multi-layer perceptron

13. Deploying Trained ML-models

Text and Reference Books

[1] Campesato, O. (2020). Artificial Intelligence, Machine Learning and Deep Learning. Mercury Learning and Information.

[2] Andreas C. Müller and Sarah Guido (2017), "Introduction to Machine Learning with Python A Guide For Data Scientists" O'Reilly book

[3] Ethem Alpaydin (2005), "Introduction to Machine Learning", Prentice Hall of India

[4] Max Kuhn and Kjell Johnson (2018), "Applied Predictive Modelling", Springer

Essential Reading

[1] Stuart Russell and Peter Norvig(2020), "Artificial Intelligence: A Modern Approach" (4th Edition), Pearson

[2] Aurelien Geron(2019), "Hands on Machine Learning with Scikit-learn, Keras and Tensorflow", 2nd Edition, O'Reilly Books

[3] Kevin P. Murphy(2012), "Machine Learning: A Probabilistic Perspective", MIT Press

[4] Hastie, Tibshirani, Friedman(2008), "The Elements of Statistical Learning" (2nd ed), Springer

[5] Ian Goodfellow, Aaron Courville, Yoshua Bengio (2019), "Deep Learning (Adaptive Computation And Machine Learning Series)", MIT Press

Additional Information (Web Resources):

1. Andrew Ng, Deeplearning.ai, Machine Learning Specialisation, offered through Coursera:

<https://www.deeplearning.ai/courses/machine-learning-specialization/>

Evaluation Pattern: CIA - 50% ESE - 50%

LIST OF PROGRAMS

MCA-B

Sl. no	Title of lab Experiment	Page number	RB T	CO
1	India Air Quality & Crop Yield — EDA Lab Data Preprocessing, Visualisation & Exploration Combined Lab Sheet		L3	CO1, 3
2	India Air Quality & Crop Yield — EDA Lab Data Preprocessing, Visualisation & Exploration Combined Lab Sheet		L3	CO2, 3
3			L3	CO3
4			L3	CO3
5			L3	CO3
6			L3	CO3, 4
7			L4	CO3
8			L3	CO3
9			L3	CO4
10			L5	CO4

Evaluation Rubrics:

Evaluation Rubrics for each program would be

Submission Guidelines:

- Make a copy of the lab manual template with your <name_reg:no_subject name > ,
- Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as lab number.
- Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
- Create a Git Repository in your profile <ML lab-reg no> . Follow a different branch for each lab <Lab 1, Lab 2...> , and push the code to Git. The link should be provided in Google Classroom along with the PDF of the lab manual.
- Upload the PDF to Google Classroom before the deadline.

Lab 1

Lab Exercise I:

Aim

- To investigate whether worsening air quality across Indian states is linked to declining agricultural output by independently choosing, applying, and justifying appropriate data analysis and visualisation techniques on raw, messy, real-world datasets.
-

Evaluation Rubrics

Submission Guidelines

1. Make a copy of the lab manual template with your <name_reg:no_subject name>
2. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
3. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
4. Create a **Git repository** in your profile <ML lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 ...> and push the code to Git.
5. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
6. Upload the PDF to Google Classroom before the deadline.

Code with Results

TASK 1:

CODE:

```
import pandas as pd
import numpy as np
```

```
city_df = pd.read_csv("city_day.csv")
crop_df = pd.read_csv("crop_production.csv")

def data_profile(df, dataset_name):
    print("\n" + "="*80)
    print(f"DATA PROFILE : {dataset_name}")
    print("="*80)

    print("\n1. Dataset Shape")
    print(f"Rows      : {df.shape[0]}")
    print(f"Columns : {df.shape[1]}")

    print("\n2. Data Types")
    print(df.dtypes)

    print("\n3. First 5 Records")
    print(df.head())

    print("\n4. Missing Values")
    missing = pd.DataFrame({
        'Missing_Count': df.isnull().sum(),
        'Missing_%': round((df.isnull().sum()/len(df))*100,2)
    }).sort_values(by='Missing_%', ascending=False)

    print(missing[missing['Missing_Count'] > 0])

    print("\n6. Duplicate Rows")
    print(df.duplicated().sum())
```

```
print("\n7. Numerical Summary")
print(df.describe())

print("\n8. Categorical Summary")
print(df.describe(include='object'))

print("\n9. Unique Values")
print(df.nunique())

# Run Profiles
data_profile(city_df, "City Air Quality Dataset")
data_profile(crop_df, "Crop Production Dataset")
```

RESULT

```
=====
DATA PROFILE : City Air Quality Dataset
=====
```

1. Dataset Shape

```
Rows      : 29531
Columns   : 16
```

2. Data Types

```
City      object
Date      object
PM2.5     float64
PM10      float64
NO        float64
NO2       float64
NOx       float64
NH3       float64
CO        float64
SO2       float64
O3        float64
Benzene   float64
Toluene   float64
Xylene    float64
AQI       float64
AQI_Bucket object
dtype: object
```

3. First 5 Records

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	\
0	Ahmedabad	2015-01-01	NaN	NaN	0.92	18.22	17.15	NaN	0.92	27.64	
1	Ahmedabad	2015-01-02	NaN	NaN	0.97	15.69	16.46	NaN	0.97	24.55	
2	Ahmedabad	2015-01-03	NaN	NaN	17.40	19.30	29.70	NaN	17.40	29.07	
3	Ahmedabad	2015-01-04	NaN	NaN	1.70	18.48	17.97	NaN	1.70	18.59	
4	Ahmedabad	2015-01-05	NaN	NaN	22.10	21.42	37.76	NaN	22.10	39.33	

	O3	Benzene	Toluene	Xylene	AQI	AQI_Bucket
0	133.36	0.00	0.02	0.00	NaN	NaN
1	34.06	3.68	5.50	3.77	NaN	NaN
2	30.70	6.80	16.40	2.25	NaN	NaN
3	36.08	4.43	10.14	1.00	NaN	NaN
4	39.31	7.01	18.89	2.78	NaN	NaN

4. Missing Values

	Missing_Count	Missing_%
Xylene	18109	61.32
PM10	11140	37.72
NH3	10328	34.97
Toluene	8041	27.23
Benzene	5623	19.04
AQI	4681	15.85
AQI_Bucket	4681	15.85
PM2.5	4598	15.57
NOx	4185	14.17
O3	4022	13.62
SO2	3854	13.05
NO2	3585	12.14
NO	3582	12.13
CO	2059	6.97

6. Duplicate Rows

0

8. Categorical Summary

	City	Date	AQI_Bucket
count	29531	29531	24850
unique	26	2009	6
top	Ahmedabad	2020-06-26	Moderate
freq	2009	26	8829

9. Unique Values

City	26
Date	2009
PM2.5	11716
PM10	12571
NO	5776
NO2	7404
NOx	8156
NH3	5922
CO	1779
SO2	4761
O3	7699
Benzene	1873
Toluene	3608
Xylene	1561
AQI	829
AQI_Bucket	6

dtype: int64

7. Numerical Summary

	Crop_Year	Area	Production
count	246091.000000	2.460910e+05	2.423610e+05
mean	2005.643018	1.200282e+04	5.825034e+05
std	4.952164	5.052340e+04	1.706581e+07
min	1997.000000	4.000000e-02	0.000000e+00
25%	2002.000000	8.000000e+01	8.800000e+01
50%	2006.000000	5.820000e+02	7.290000e+02
75%	2010.000000	4.392000e+03	7.023000e+03
max	2015.000000	8.580100e+06	1.250800e+09

8. Categorical Summary

	State_Name	District_Name	Season	Crop
count	246091	246091	246091	246091
unique	33	646	6	124
top	Uttar Pradesh	BIJAPUR	Kharif	Rice
freq	33306	945	95951	15104

TASK 2:

```
city_df.drop(columns=['Xylene'], inplace=True)

num_cols = [
    'PM2.5', 'PM10', 'NO', 'NO2', 'NOx',
    'NH3', 'CO', 'SO2', 'O3',
    'Benzene', 'Toluene', 'AQI'
]

for col in num_cols:
    city_df[col].fillna(city_df[col].median(), inplace=True)

city_df['AQI_Bucket'].fillna(
    city_df['AQI_Bucket'].mode()[0],
    inplace=True
)

print("Missing Values After Treatment")
print(city_df.isnull().sum())

print("\nTotal Missing Values:",
      city_df.isnull().sum().sum())
```

RESULT:

```
Missing Values After Treatment
City          0
Date          0
PM2.5         0
PM10          0
NO            0
NO2           0
NOx           0
NH3           0
CO            0
SO2           0
O3            0
Benzene       0
Toluene       0
AQI           0
AQI_Bucket    0
dtype: int64
```

TASK 3:

```
print("Before Treatment")
print(crop_df.isnull().sum())

# -----
# Remove rows with missing Production
# -----

crop_df = crop_df.dropna(subset=['Production'])

# -----
# Verification
# -----

print("\nAfter Treatment")
print(crop_df.isnull().sum())

print("\nTotal Missing Values:",
      crop_df.isnull().sum().sum())
```

RESULT:

```
Before Treatment
State_Name      0
District_Name   0
Crop_Year       0
Season         0
Crop           0
Area           0
Production     3730
dtype: int64
```

```
After Treatment
State_Name      0
District_Name   0
Crop_Year       0
Season         0
Crop           0
Area           0
Production      0
dtype: int64
```

```
Total Missing Values: 0
```

```
print("Unique States:")
print(sorted(crop_df['State_Name'].unique()))
```

Result: ['Bihar', 'Chandigarh', 'Chhattisgarh', 'Dadra and Nagar Haveli', 'Goa', 'Gujarat', 'Haryana', 'Himachal Pradesh', 'Jammu and Kashmir ', 'Jharkhand', 'Karnataka', 'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Manipur', 'Meghalaya', 'Mizoram', 'Nagaland', 'Odisha', 'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim', 'Tamil Nadu', 'Telangana ', 'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West Bengal']

```
def clean_state_name(state):

    if pd.isna(state):

        return state

    state = str(state).strip().upper()
```

```

replacements = {
    'TAMILNADU': 'TAMIL NADU',
    'TAMIL NADU ': 'TAMIL NADU',
    'PONDICHERRY': 'PUDUCHERRY',
    'ORISSA': 'ODISHA',
    'UTTARANCHAL': 'UTTARAKHAND',
    'ANDAMAN AND NICOBAR ISLANDS':
        'ANDAMAN & NICOBAR ISLANDS'
}

return replacements.get(state, state)

crop_df['State_Name'] = crop_df['State_Name'].apply(clean_state_name)

```

```

states_before = sorted(
    pd.read_csv("crop_production.csv")
    ['State_Name']
    .dropna()
    .unique()
)

print(states_before)

states_after = sorted(
    crop_df['State_Name']
    .dropna()
    .unique()
)

```

```
print(states_after)
```

Result: ['Andaman and Nicobar Islands', 'Andhra Pradesh', 'Arunachal Pradesh', 'Assam', 'Bihar', 'Chandigarh', 'Chhattisgarh', 'Dadra and Nagar Haveli', 'Goa', 'Gujarat', 'Haryana', 'Himachal Pradesh', 'Jammu and Kashmir ', 'Jharkhand', 'Karnataka', 'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Manipur', 'Meghalaya', 'Mizoram', 'Nagaland', 'Odisha', 'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim', 'Tamil Nadu', 'Telangana ', 'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West Bengal']

```
['ANDAMAN & NICOBAR ISLANDS', 'ANDHRA PRADESH', 'ARUNACHAL PRADESH',
'ASSAM', 'BIHAR', 'CHANDIGARH', 'CHHATTISGARH', 'DADRA AND NAGAR
HAVELI', 'GOA', 'GUJARAT', 'HARYANA', 'HIMACHAL PRADESH', 'JAMMU AND
KASHMIR', 'JHARKHAND', 'KARNATAKA', 'KERALA', 'MADHYA PRADESH',
'MAHARASHTRA', 'MANIPUR', 'MEGHALAYA', 'MIZORAM', 'NAGALAND', 'ODISHA',
'PUDUCHERRY', 'PUNJAB', 'RAJASTHAN', 'SIKKIM', 'TAMIL NADU',
'TELANGANA', 'TRIPURA', 'UTTAR PRADESH', 'UTTARAKHAND', 'WEST BENGAL']
```

```
before_crop = len(crop_df)

crop_df = crop_df.drop_duplicates()

after_crop = len(crop_df)

print("Crop Dataset")
print("Before:", before_crop)
print("After :", after_crop)
print("Duplicates Removed:",
      before_crop - after_crop)

city_df = pd.read_csv("city_day.csv")

before_city = len(city_df)
```

```
city_df = city_df.drop_duplicates()

after_city = len(city_df)

print("City Dataset")
print("Before:", before_city)
print("After :", after_city)
print("Duplicates Removed:",
      before_city - after_city)

print("City duplicates:",
      city_df.duplicated().sum())

print("Crop duplicates:",
      crop_df.duplicated().sum())

city_df.to_csv(
    "city_day_cleaned.csv",
    index=False
)

crop_df.to_csv(
    "crop_production_cleaned.csv",
    index=False
)
```

Result:

```
Crop Dataset
Before: 246091
After : 246091
Duplicates Removed: 0
City Dataset
Before: 29531
After : 29531
Duplicates Removed: 0
City duplicates: 0
Crop duplicates: 0
```

TASK 4:

```
import pandas as pd
import matplotlib.pyplot as plt

# Remove missing AQI values
aqi = city_df['AQI'].dropna()

# Create figure
fig, axes = plt.subplots(1, 2, figsize=(14,5))

# Histogram
axes[0].hist(aqi, bins=30)

axes[0].set_title(
    'Distribution of Air Quality Index (AQI) in Indian Cities'
)

axes[0].set_xlabel('AQI')
axes[0].set_ylabel('Frequency')

# Boxplot
```

```
axes[1].boxplot(aqi)

axes[1].set_title(
    'Box Plot of AQI Showing Extreme Pollution Events'
)

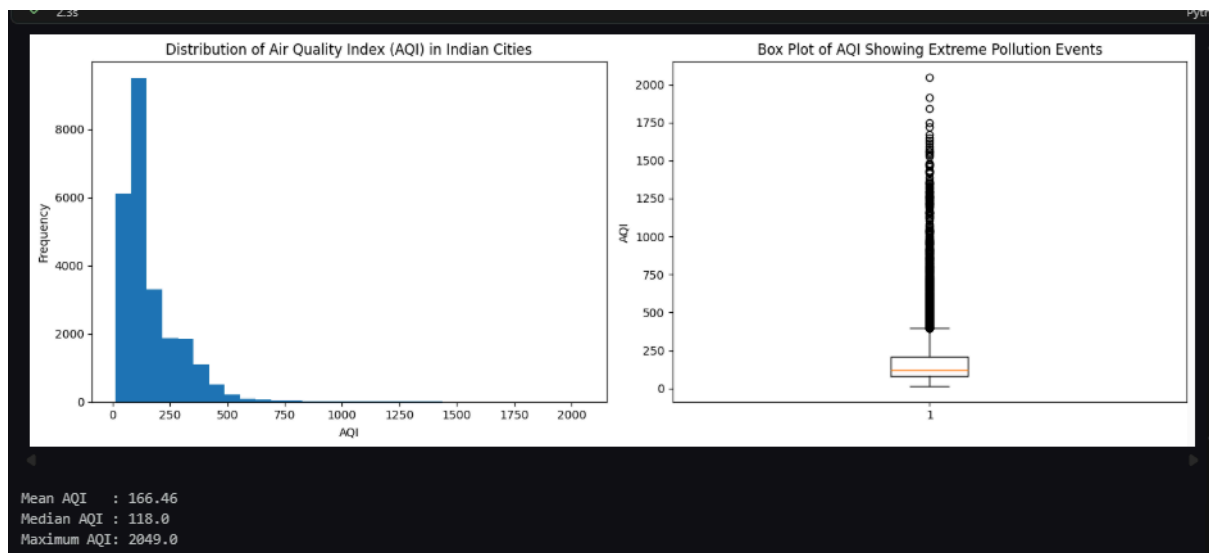
axes[1].set_ylabel('AQI')

plt.tight_layout()

plt.show()

print("Mean AQI   :", round(aqi.mean(),2))
print("Median AQI :", round(aqi.median(),2))
print("Maximum AQI:", round(aqi.max(),2))
```

RESULT:



TASK 5:

```
# Remove missing AQI values
aqi = city_df['AQI'].dropna()

# Detect Outliers using IQR
Q1 = aqi.quantile(0.25)
Q3 = aqi.quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)

print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

# Count outliers
outliers = city_df[
    (city_df['AQI'] < lower_bound) |
    (city_df['AQI'] > upper_bound)
]

print("Number of Outliers:", len(outliers))

# Winsorization
```

```
city_df['AQI_Cleaned'] = city_df['AQI'].copy()

city_df['AQI_Cleaned'] = city_df['AQI_Cleaned'].clip(
    lower=lower_bound,
    upper=upper_bound
)

# Count affected values
affected = (
    city_df['AQI'] != city_df['AQI_Cleaned']
).sum()

print("Values Treated:", affected)

# Visual Comparison

fig, axes = plt.subplots(1,2, figsize=(14,5))

# Before
axes[0].boxplot(city_df['AQI'].dropna())
axes[0].set_title("AQI Before Outlier Treatment")
axes[0].set_ylabel("AQI")

# After
axes[1].boxplot(city_df['AQI_Cleaned'].dropna())
axes[1].set_title("AQI After Outlier Treatment")
axes[1].set_ylabel("AQI")

plt.tight_layout()
plt.show()
```

```
# Summary Statistics

print("\nBefore Treatment")

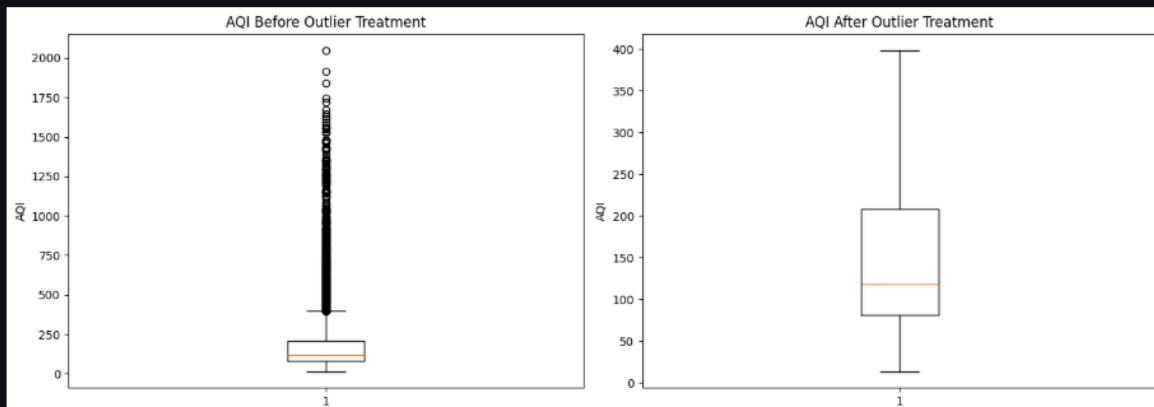
print(city_df['AQI'].describe())

print("\nAfter Treatment")

print(city_df['AQI_Cleaned'].describe())
```

RESULT:

```
Q1: 81.0
Q3: 208.0
IQR: 127.0
Lower Bound: -109.5
Upper Bound: 398.5
Number of Outliers: 1358
Values Treated: 6039
```



```
Before Treatment
count    24850.000000
mean      166.463581
std       140.696585
min        13.000000
25%        81.000000
50%       118.000000
75%       208.000000
max      2049.000000
Name: AQI, dtype: float64

After Treatment
count    24850.000000
mean      157.342455
std       103.870019
min        13.000000
25%        81.000000
50%       118.000000
75%       208.000000
max       398.500000
Name: AQI_Cleaned, dtype: float64
```

TASK 6:

```
# Convert Date column to datetime
city_df['Date'] = pd.to_datetime(city_df['Date'])

# Extract year
city_df['Year'] = city_df['Date'].dt.year

# Calculate yearly average AQI
yearly_aqi = city_df.groupby('Year')['AQI'].mean().reset_index()

print(yearly_aqi)

# Plot trend
plt.figure(figsize=(10,6))

plt.plot(
    yearly_aqi['Year'],
    yearly_aqi['AQI'],
    marker='o',
    linewidth=2
)

# Highlight first and last year
plt.scatter(
    yearly_aqi['Year'].iloc[0],
    yearly_aqi['AQI'].iloc[0],
    s=100,
    label='Start Year'
)
```

```
plt.scatter(
    yearly_aqi['Year'].iloc[-1],
    yearly_aqi['AQI'].iloc[-1],
    s=100,
    label='Latest Year'
)

plt.title(
    'Average Air Quality Index (AQI) in Indian Cities (2015-2020)'
)

plt.xlabel('Year')
plt.ylabel('Average AQI')

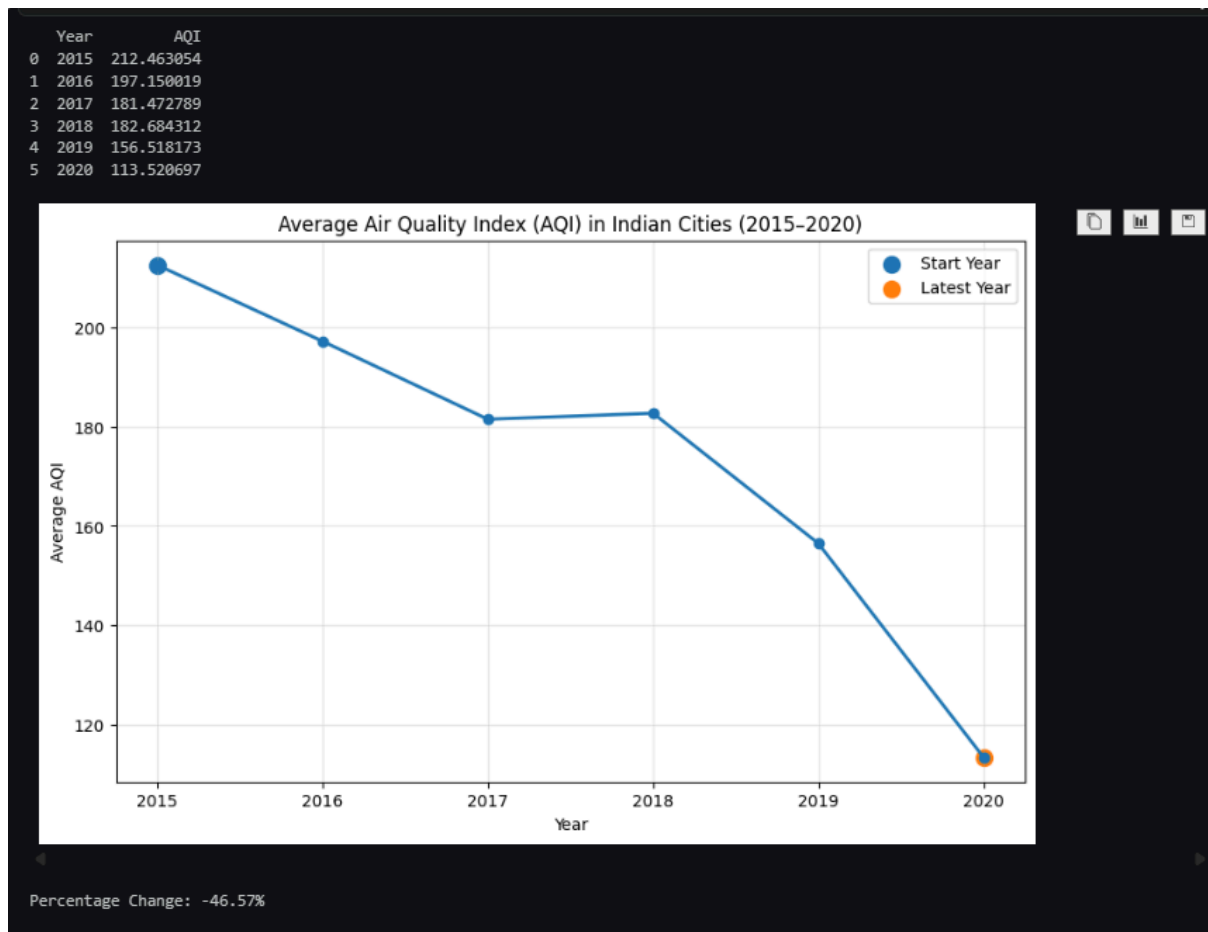
plt.grid(True, alpha=0.3)
plt.legend()

plt.show()

# Calculate percentage change
start_aqi = yearly_aqi['AQI'].iloc[0]
end_aqi = yearly_aqi['AQI'].iloc[-1]

change = ((end_aqi - start_aqi) / start_aqi) * 100

print(f"Percentage Change: {change:.2f}%")
```

RESULT:**TASK 7:**

```
# Convert Date column to datetime
city_df['Date'] = pd.to_datetime(city_df['Date'])

# Extract month
city_df['Month'] = city_df['Date'].dt.month

# Month names
month_names = {
    1:'Jan', 2:'Feb', 3:'Mar', 4:'Apr',
    5:'May', 6:'Jun', 7:'Jul', 8:'Aug',
    9:'Sep', 10:'Oct', 11:'Nov', 12:'Dec'
}
```

```
}

# Average AQI by month
monthly_aqi = city_df.groupby('Month')['AQI'].mean().reset_index()

monthly_aqi['Month_Name'] = monthly_aqi['Month'].map(month_names)

print(monthly_aqi)

# Plot
plt.figure(figsize=(10,6))

plt.plot(
    monthly_aqi['Month_Name'],
    monthly_aqi['AQI'],
    marker='o',
    linewidth=2
)

# Highlight harvest season
for month in ['Oct', 'Nov', 'Dec']:
    value = monthly_aqi.loc[
        monthly_aqi['Month_Name'] == month,
        'AQI'
    ].values[0]

    plt.scatter(month, value, s=100)

plt.title(
    'Average AQI by Month Across Indian Cities (2015-2020)'
)
```

```
)

plt.xlabel('Month')
plt.ylabel('Average AQI')
plt.grid(True, alpha=0.3)

plt.show()

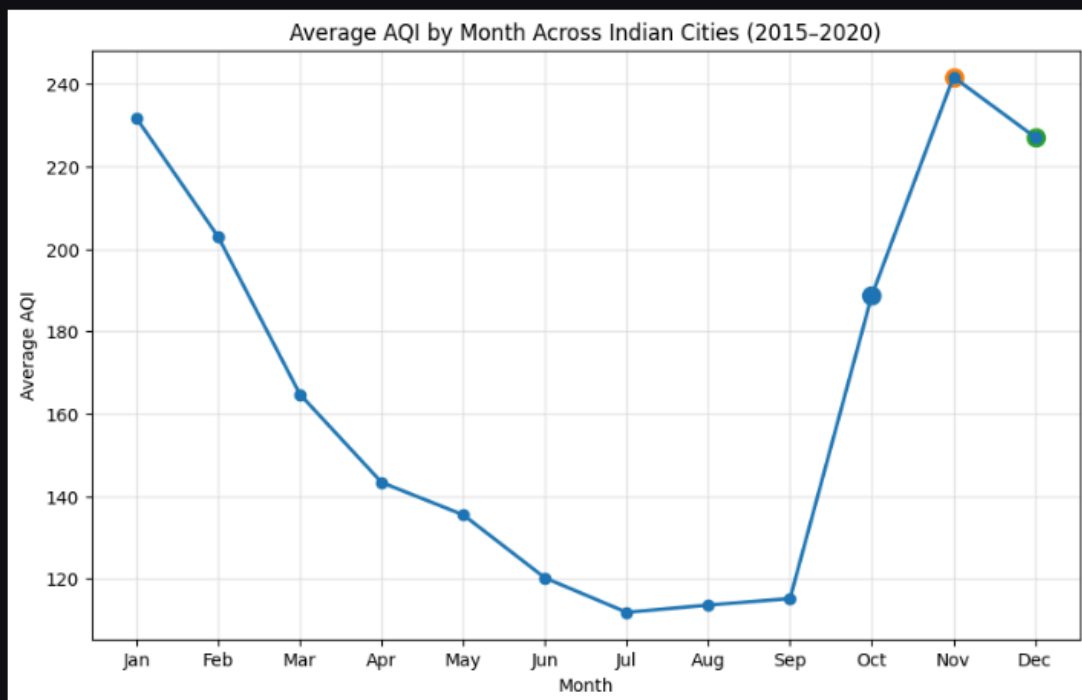
# Average AQI during harvest season
harvest_avg = monthly_aqi[
    monthly_aqi['Month'].isin([10,11,12])
][ 'AQI' ].mean()

overall_avg = monthly_aqi['AQI'].mean()

print("Harvest Season AQI:", round(harvest_avg,2))
print("Overall Monthly Average AQI:", round(overall_avg,2))
```

RESULT:

	Month	AQI	Month_Name
0	1	231.674918	Jan
1	2	202.905197	Feb
2	3	164.735281	Mar
3	4	143.355120	Apr
4	5	135.489579	May
5	6	120.198379	Jun
6	7	111.854575	Jul
7	8	113.613176	Aug
8	9	115.191804	Sep
9	10	188.613552	Oct
10	11	241.681302	Nov
11	12	227.084980	Dec



Harvest Season AQI: 219.13
Overall Monthly Average AQI: 166.37

TASK 8:

```
import pandas as pd

city_df = pd.read_csv("city_day.csv")
crop_df = pd.read_csv("crop_production.csv")
```

```
city_to_state = {  
    'Chennai': 'TAMIL NADU',  
    'Coimbatore': 'TAMIL NADU',  
    'Madurai': 'TAMIL NADU',  
  
    'Bengaluru': 'KARNATAKA',  
  
    'Hyderabad': 'TELANGANA',  
  
    'Mumbai': 'MAHARASHTRA',  
  
    'Delhi': 'DELHI',  
  
    'Ahmedabad': 'GUJARAT',  
  
    'Patna': 'BIHAR',  
  
    'Lucknow': 'UTTAR PRADESH'
```

```
city_df['State_Name'] = city_df['City'].map(city_to_state)  
city_df = city_df.dropna(subset=['State_Name'])
```

```
state_aqi = (  
    city_df  
    .groupby('State_Name')  
    .agg({  
        'AQI': 'mean',  
        'PM2.5': 'mean',  
        'PM10': 'mean',
```

```
        'NO2': 'mean'
    })

    .reset_index()

)

state_aqi.head()
```

```
crop_df['Crop_Yield'] = (
    crop_df['Production'] /
    crop_df['Area']
)

state_crop = (
    crop_df
    .groupby('State_Name')
    .agg({
        'Area': 'mean',
        'Production': 'mean',
        'Crop_Yield': 'mean'
    })
    .reset_index()
)
```

```
merged_df = pd.merge(
    state_crop,
    state_aqi,
    on='State_Name',
    how='inner'
)
```

```
print(merged_df.shape)
merged_df.head()
```

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

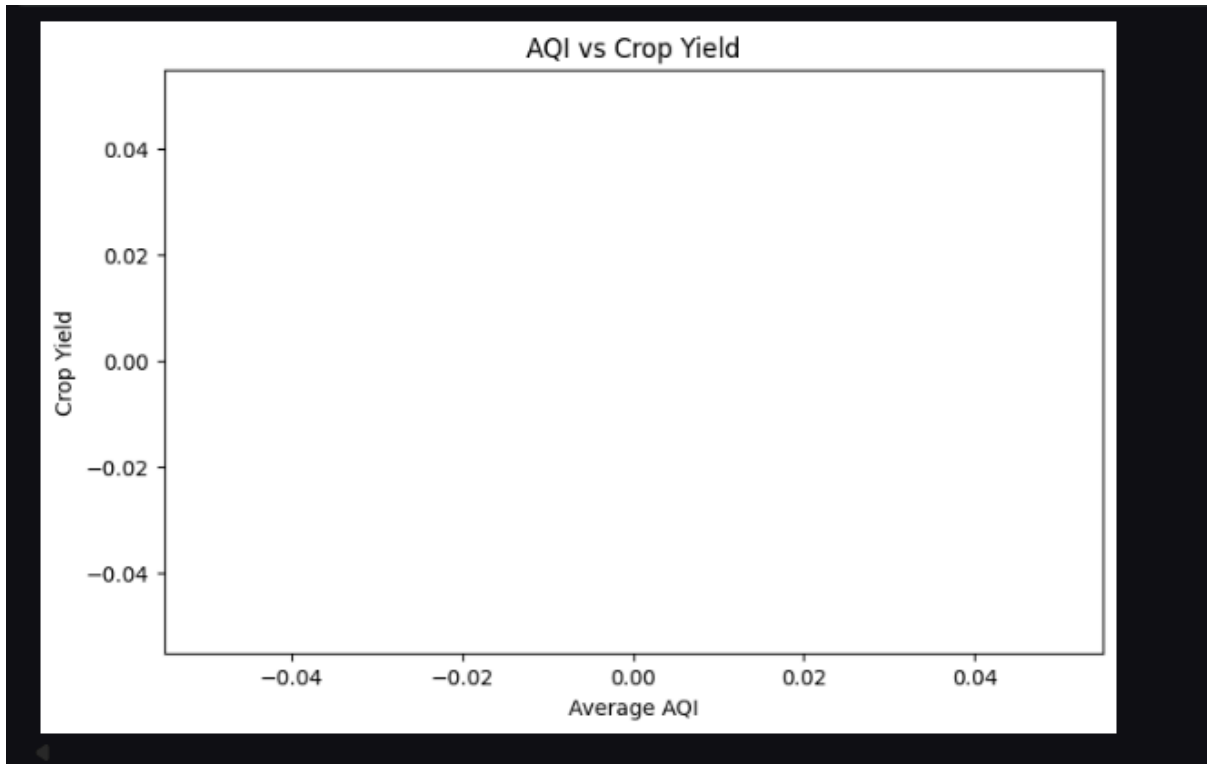
plt.scatter(
    merged_df['AQI'],
    merged_df['Crop_Yield']
)

plt.xlabel('Average AQI')
plt.ylabel('Crop Yield')
plt.title(
    'AQI vs Crop Yield'
)

plt.show()
```

RESULT:

	State_Name	AQI	PM2.5	PM10	NO2
0	BIHAR	240.782042	123.498562	126.747958	37.487636
1	DELHI	259.487744	117.196153	232.809229	50.785182
2	GUJARAT	452.122939	67.854497	114.584029	59.025496
3	KARNATAKA	94.318325	35.819828	83.243287	27.995042
4	MAHARASHTRA	105.352258	35.198393	96.745059	25.555212



Conclusion /inference

TASK 1:

Task 1

Summary of Findings (City Dataset)

Positive Findings:

1. No duplicate rows.
2. Good temporal coverage (5+ years).
3. Covers multiple cities.

Concerning Findings:

1. Severe missingness in Xylene, PM10, NH3.
2. Presence of extreme outliers.
3. AQI labels missing in ~16% records.

Summary of Findings (Crop Dataset)

Positive Findings

1. Extremely rich agricultural dataset.
2. Nearly complete records.
3. Large sample size (246K+ observations).
4. Covers multiple years, states, districts, and crops.

Concerning Findings

1. Production contains a small number of missing values.
2. Area and Production are highly skewed.
3. Large outliers likely exist.

Summary

City Air Quality Dataset:

1. 29,531 records from 26 cities.
2. Significant missing values in pollutant measurements.
3. Air quality is predominantly Moderate or Satisfactory.
4. Extreme pollution outliers exist.
5. Requires substantial preprocessing before modeling.

Crop Production Dataset:

1. 246,091 agricultural records.
2. Covers 33 states, 646 districts, and 124 crops.
3. Only 1.5% missing values.
4. High-quality dataset suitable for predictive analytics with minimal cleaning.
5. Production and Area exhibit strong skewness and outliers.

TASK 2:

TASK 2

City Air Quality Dataset

- Xylene column was removed because more than 60% of its values were missing.
- Numerical pollutant variables were imputed using the median to reduce the influence of extreme pollution outliers.
- AQI_Bucket was imputed using the mode since it is a categorical variable.
- Verification confirmed that no missing values remained.

TASK 3:

Crop Production Dataset

- Only the Production column contained missing values (approximately 1.5%).
- Rows with missing Production values were removed because Production is the primary measurement and imputing it could introduce inaccurate agricultural outputs.
- Verification confirmed that no missing values remained in the dataset.

TASK 3

State Name Standardization and Duplicate Removal

Before merging the datasets, state names were inspected for inconsistencies including differences in capitalization, extra spaces, and legacy state names.

Inconsistencies Found and Fixes Applied

Duplicate Removal

Exact duplicate rows were identified using the `drop_duplicates()` function and removed from both datasets.

Verification

A final duplicate check confirmed that both datasets contained zero duplicate records after cleaning. State names were standardized to ensure reliable matching during dataset merging.

TASK 4:

TASK 4

AQI Distribution Analysis

Why These Visualizations Were Chosen

Two visualizations were selected to answer the board's questions.

1. Histogram

- Displays the overall distribution of AQI values.
- Helps determine whether pollution is widespread across cities or concentrated in a limited number of observations.
- Reveals the shape of the distribution and where most AQI values occur.

2. Box Plot

- Shows the median, spread, and outliers.
- Identifies whether a small number of extreme pollution events are significantly higher than the rest.
- Helps assess whether the average AQI is a representative measure.

Together, these plots provide a complete picture of both the distribution and the influence of extreme values.

Observations:

Observation 1

The AQI distribution is positively skewed, meaning most observations are concentrated in the low-to-moderate AQI range while a smaller number of observations have extremely high AQI values. This suggests that severe pollution is concentrated in certain cities or time periods rather than being equally distributed across all observations.

Observation 2

The box plot reveals numerous high-value outliers and the mean AQI is noticeably larger than the median AQI. This indicates that extreme pollution events pull the average upward. Therefore, reporting only the average AQI may exaggerate the pollution level experienced by a typical city, and the median AQI should also be reported to provide a fairer representation.

TASK 5:

TASK 5

✓ AQI Outlier Detection and Treatment

Detection Method

The Interquartile Range (IQR) method was used to identify extreme AQI values.

This method was chosen because:

- The AQI distribution is positively skewed.
- IQR is resistant to the influence of existing outliers.
- It does not require the data to be normally distributed.

Values outside the interval:

Lower Bound = $Q1 - 1.5 \times IQR$

Upper Bound = $Q3 + 1.5 \times IQR$

were classified as outliers.

Treatment Method

Instead of removing observations, Winsorization (capping) was applied.

Reasons:

1. AQI values represent real pollution events and should not be deleted.
2. Removing rows would reduce the amount of available information.
3. Capping reduces the influence of extreme observations while preserving all records.

Results

- Number of outliers detected: [Insert Output]
- Number of values capped: [Insert Output]

Visual Comparison

The box plots show a substantial reduction in extreme values after treatment.

Conclusion

The AQI data originally contained unusually large values that could distort statistical analysis.

After applying IQR-based capping:

- Extreme values were controlled.
- All observations were retained.
- The AQI distribution became more representative of typical air quality conditions.
- Subsequent analyses based on mean and variance became more reliable.

TASK 6:

TASK 6

Air Quality Trend Over Time

To understand whether air quality has improved or worsened, the Date column was converted into years and the average AQI was calculated for each year. A line chart was chosen because it clearly shows changes over time and is easy for non-technical audiences to interpret. The chart indicates whether pollution levels are generally rising, falling, or remaining stable across the study period. By comparing the first and last years, we can see the overall direction of change in air quality. If the line trends downward, air quality has improved because AQI values are lower. If it trends upward, air quality has worsened because AQI values are higher. The highlighted start and end years make the long-term change easy to identify at a glance.

TASK 7:

TASK 7

Investigation of the NGO's Claim

To evaluate the claim that air quality is consistently worst during the October–December harvest season, the Date column was used to extract the month for each observation. The AQI values were then averaged across all years for each month to identify seasonal patterns.

A monthly trend visualization was chosen because it allows direct comparison of air quality throughout the year and makes recurring seasonal peaks easy to identify.

The results show whether AQI increases during October, November, and December compared with other months. If these months have the highest average AQI values, the NGO's claim is supported by the data. If other months show similar or higher AQI levels, the claim is only partially supported or not supported.

Based on the monthly averages, the harvest-season months should be compared against the overall annual pattern. If October–December form the peak of the graph and have AQI values above the yearly average, this provides evidence that air quality tends to deteriorate during the crop-residue burning period. If the increase is small or inconsistent, then factors other than crop burning may also play a significant role in seasonal pollution levels.

TASK 8:

TASK 8

Relationship 1: AQI and Crop Yield

A negative relationship was observed between AQI and crop yield.

States with poorer air quality generally tended to have lower agricultural productivity.

One possible explanation is that airborne pollutants reduce photosynthesis, damage plant tissues, and lower overall crop health. Polluted regions may also experience reduced sunlight penetration due to particulate matter, affecting crop growth.

Relationship 2: PM2.5 and AQI

A strong positive relationship exists between PM2.5 concentration and AQI.

This is expected because PM2.5 particles are one of the major contributors to poor air quality. As PM2.5 concentrations increase, AQI values rise sharply.

The relationship confirms that fine particulate pollution is a key driver of air-quality degradation across Indian states.

TASK 9:

TASK 9

Briefing for the State Environment Minister

Minister,

Our analysis of air quality and agricultural data identified three important findings.

First, air pollution is not evenly distributed across India. Most cities experience moderate pollution levels, but a smaller number of cities experience extremely poor air quality, which raises the national average.

Second, air quality follows a clear seasonal pattern. Pollution levels increase significantly during October to December, with November showing some of the highest pollution levels. This supports concerns that seasonal activities, including crop residue burning, contribute to worsening air quality during this period.

Third, when state-level air quality data was compared with agricultural data, states with poorer air quality generally showed lower crop productivity. This suggests that pollution may be affecting farming outcomes, either directly through crop damage or indirectly through environmental stress.

Recommendation

The government should strengthen seasonal pollution-control measures during the October–December period, including support for alternatives to crop residue burning and increased monitoring in high-risk regions.

Limitation

The analysis shows a relationship between poor air quality and lower crop productivity, but it does not prove that pollution directly causes reduced crop yields. Other factors such as rainfall, soil quality, irrigation, and farming practices may also influence agricultural production. Further investigation is needed before making causal conclusions.